МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И
КОМПЬЮТЕРНОЙ ТЕХНИКИ

ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине

«АРХИТЕКТУРА КОМПЬЮТЕРА»

Вариант :

alg | cisc | harv | hw | tick | binary

trap | port | cstr | prob1 | vector

Выполнил:

Софьин Вячеслав Евгеньевич

Проверил:

Пенской Александр Владимирович

г. Санкт-Петербург, 2026

Содержание

1. Алгоритмическия язык . . . 3
   1.1. Выражения . . . 3
   1.2. Абстрактное синтаксическое дерево . . . 4
   1.3. Семантический анализ . . . 5
   1.4. Конвенция названий . . . 6
   1.5. Семантический анализ . . . 6
2. Организация памяти . . . 7
   2.1. Описание архитектуры . . . 7
   2.2. Внешние устройства . . . 8
   2.3. Использования памяти транслятором . . . 9
3. Поток компиляции . . . 11
4. Система команд . . . 13
   4.1. Инструкции с n операторов . . . 13
   4.2. Инструкции переходов . . . 13
   4.3. Описание операнда . . . 13
   4.4. Режим операнда . . . 13
   4.5. Коды операторов: . . . 14
5. Транслятор . . . 18
6. Управляющее устройство . . . 19
7. Тракт данных . . . 20
8. Векторные операции . . . 23
   8.1. Обшая информация . . . 23
   8.2. Сложение векторов . . . 23
   8.3. Перемножение матриц . . . 24

1. Алгоритмическия язык

1.1. Выражения

```
literal ::= number | string | char | "true" | "false" ;
binary-operator ::= "+" | "-" | "*" | "/" | "%"
          | "<<" | ">>"
          | "&" | "|" | "^"
          | "&&" | "||"
          | "==" | "!=" | "<" | "<=" | ">" | ">=" ;
assignment-operator ::= "=" | "+=" | "-="
          | "*=" | "/=" | "&=" | "|=" | "^=" ;

value ::= literal
          | identifier
          | "this"
          | "(" expression ")"
          | "new" identifier "(" ")"
          | "new" identifier "[" number "]"
          | "[" [ literal { "," literal } ] "]" ;
box ::= identifier
         | value "." identifier
         | value "[" expression "]"
         | value "[" expression ":" number "]" ;
expression ::= value { postfix-operator }
         | prefix-operator expression
         | expression binary-operator expression
         | box assignment-operator expression ;

prefix-operator ::= "-" | "!" | "~" | "++" | "--" ;
postfix-operator ::= "++" | "--"
                   | "." identifier [ "(" arguments ")" ]
                   | "[" expression "]"
                   | "[" expression ":" number "]" ;

arguments ::= [ expression { "," expression } ] ;
```

1.2. Абстрактное синтаксическое дерево

```
type ::= "int" | "bool" | "char" | "void"
    | identifier
    | type "[" number "]" ;

program ::= { declaration | function | class } ;
declaration ::= type identifier [ "=" expression ] ";" ;

statement ::=
      expression ";"
    | declaration ";"
    | "return" [ expression ] ";"
    | "break" ";"
    | "continue" ";"
    | if-statement
    | while-statement
    | for-statement
    | "{" { statement } "}" ;

argument ::= type identifier ;
function ::= type identifier "(" [ argument { "," argument } ] ")"
"{" { statement } "}" ;

class ::= "class" identifier "{" { declaration | function }
"}" ;

if-statement ::= "if" "(" expression ")" "{" { statement } "}"
      { "else if" "(" expression ")" "{" { statement } "}" }
      [ "else" "{" { statement } "}" ] ;

while-statement ::= "while" "(" expression ")" "{" { statement }
"}" ;

for-initializer ::= declaration | expression ;
for-statement ::= "for" "(" [ for-initializer ] ";" [ expression ]
";" [ expression ] ")" "{" { statement } "}" ;
```

1.3. Семантический анализ

1.3.1. Типизация

Типизация строгая статическая.

Типы делятся на два вида: примитивы и ссылочные. К примитивам относятся int, bool, char. Ссылочными типами данных являются одномерные массивы и классы.

Int, bool, char и массивы можно задавать литералами.

Массивы имеют константную длину. Это нужно, чтобы статический анализ хоть немного работал. Массивами (и слайсы) длины 4 расширены всем набором арифметических операций. Они выполняются с помощью векторного блока.

1.3.2. Стратегия вычислений

Стратегия вычислений - вызов по значению, аргументы вычисляются до вызова функций или методов слева направо, и копируются на стек. Внутри функции можно читать параметры. Так же можно менять, но изменяться он только в области видимости этой функции. Можно передавать ссылочные типы; содержимое того, на что они указывают можно менять. В методах всегда скрыто от программиста первым передаётся аргумент, соответствующий конкретному экземпляру класса.

1.3.3. Области видимости

Область видимости лексическая. Оформляется через фигурные скобки. Локальные переменные из вложенных областей видимости недоступны и считаются неопределёнными. Переменные из области видимости предка доступны транзитивно.

Вся программа должна находиться в одном файле.

Есть локальные, глобальные переменные. Локальные переменные определяются в областях видимости. Глобальные переменные - вне областей видимости.

Есть поля. Поля определяются внутри класса. Все поля публичные - доступны всем, кто имеет ссылку на объект.

1.4. Конвенция названий

Точкой входа в программу является функция с именем «Main».

Обработчики прерываний записываются как функции с именем «interrupt{}», с заполненым номером вектора

Для взаимодествия с внешними устройствами есть 4 функции «iin», «cin», «iout», «cout» для соответственного чтения или записи чисел или символов.

1.5. Семантический анализ

Алгоритм работает в два прохода: сначала собирает определения, а затем проверяет высказывания и выражения.

Алгоритм проверяет

- не использование повторяющихся имен классов, методов, функций, глобальных переменных
- наличие точки входа
- существование типов
- существование переменных в конкретной области видимости
- равенство типов левого и правого операнда
- доступ к арифметическим операциям и операциям сравнения есть только у int и массивов int длины 4
- доступ к операция И и ИЛИ только для bool
- условие if, while должно быть bool
- гарантию возврата из функции
- соответствие типа возвращаемого значение и типа функции
- использование this, только в методах
- и т. д.

2. Организация памяти

2.1. Описание архитектуры

Реализуется гарвардская архитектура.

Есть аппаратно-реализованный стек, для сохранения адресов возврата. Считаю, что чтение и запись происходит за 1 такт.

Память программы основана на словах в 32 бита. Поддерживает адресацию по 64 битному адресу. Доступ к ним слова в памяти программы последовательный с возможностью условных и безусловных переходов, восстановления адреса из стека. В этой памяти последовательно хранятся все инструкции и процедуры.

Есть аппаратно-реализованная таблица векторов прерываний. Поддерживает хранение до 8 векторов. Считаю, что доступ к ним мгновенный.

Память данных основана на словах в 256 бит. Есть дополнительная схемотехника позволяющая выборочно читать и изменять четверти (64 бит) этого слова. Считаю, что чтение и запись происходят за 1 такт. Чтение четверти работает также за 1 такт. Запись четверти требует двух тактов (первый для чтения всего слова, второй для изменения нужной четверти и записи). Поддерживает адресацию по 64 битному адресу. Адресация идёт по четвертям, а не по самим словам. В этой памяти хранятся данные по усмотрению программиста.

Программисту доступны 8 регистров данных, 8 адресных регистров, память данных, шина внешних устройств.

Считаю, что вектора прерываний и память программы неизменяемы в ходе выполнения программы.

Доступно 9 вариантов адресации.

- 32 битная константа, есть расширение знака
- регистры данных

- адресные регистры
- косвенное обращение к памяти по значению адресного регистра
- косвенное обращение к памяти по значению адресного регистра с его предектрементацией; для этой операции предусмотрена аппаратная реализация
- косвенное обращение к памяти по значению адресного регистра с его постинкрементацией; для этой операции предусмотрена аппаратная реализация
- косвенное обращение к памяти по значению адресного регистра с константным смещением
- косвенное обращение к памяти по значению адресного регистра с смещением по значения регистра данных (есть поддержка D5, D6, D7)
- косвенное обращение к памяти по 32 битной константе

Константы для соответствующих вариантов адресации лежат в памяти программы.

2.2. Внешние устройства

Внешние устройства и процессор связаны общей шиной. Есть линии данных, приоритета устройства, наличия прерывания и вектора прерывания. Устройства работают кооперативно. Устройства не могут выставлять наличие прерывания и записывать свой вектор, если другое устройство уже выставило наличие прерывания. Если устройство видит, что есть другое устройство с приоритетом выше, то оно должно уступить ему.

Прерывание снимается самим устройством после взаимодействия портом данных устройства.

Есть 4 устройства: ввод чисел, ввод символов, вывод числе и вывод символов. Они занимают 6 портов.

1. Чтение ввода чисел
2. Чтение или запись вектора прерываний ввода чисел

3. Чтение ввода символов
4. Чтение или запись вектора прерываний ввода символов
5. Запись в вывод чисел
6. Запись в вывод символов

2.3. Использования памяти транслятором

Транслятор моего языка программирования в начале памяти данных хранит статические данные. С 1000-го адреса начинается куча. Очистка этой области не предусмотрена. В конце памяти сверху-вниз (по направлению убывания адресов) растёт стек.

Литералы неизменяемые. К литералам относятся всё напрямую записанные данные к программе. Все литералы хранятся в статических данных. Загружаются в переменные при необходимости.

На этапе компиляции под глобальные переменные выделяется места в статической памяти. При инициализации программы, эти места заполняются нужными значениями. В процессе выполнения они могут изменяться.

Литералы и глобальные переменные хранятся вперемешку.

На стеке хранятся локальные переменные. Сюда попадают параметры функций и методов, а также вытесненные виртуальные регистры.

Int, char, bool занимают и выравниваются по четверти слова памяти.

Ссылочные типы данных хранят свои данные в куче.

Начало массива выравнивается по слову. Данные хранятся последовательно друг за другом.

Объекты выделяют под себя место необходимое для хранения всех своих полей.

Константы в памяти программы используется нуждами компилятора для записи адресов литералов, глобальных переменных и прочих констант для смещений и начального состояния.

Перед вызовы функций, методов оформлены по Cdecl, а именно перед вызовом параметры кладутся на стек в обратном порядке. При вызове сначала выполняется пролог, сохраняющий регистры (D1, D2, D3, D4, D5, A0, A1, A2) и оформляющий stack frame. В конце вызова состояние «до» восстанавливается.

При прерываниях дополнительно сохраняется и восстанавливается NZCV.

Следующие регистры зарезервированы для

- D0 - возвращение результата
- D6 - восстановление из стека регистра данных
- D7 - восстановление из стека регистра данных
- A3 - восстановление из стека адресного регистра
- A4 - восстановление из стека адресного регистра
- A5 - указатель на кучу
- A6 - указатель на stack frame
- A7 - указатель на стек

3. Поток компиляции

1. Строим абстрактное синтаксическое дерево
2. Избавляемся от синтаксического сахара
3. Проводим семантический анализ
4. Строим высокоуровневое промежуточное представление. Оно оперирует бесконечным количеством виртуальных регистров и построением по синтаксическому дереву блок схемы из высокоуровневых инструкций. Блок схема состоит из множества блоков инструкций, в конца каждого блока обязателена терминальная инструкция. Терминальная инструкция может переадресовывать к другому блоку. На этом этапе выражения рекурсивно рабиваются по «Three-address code».
5. Строим низкоуровневое промежуточное представление. Сначала переводим высокоуровневые инструкции в набор низкоуровневых. Также тут занимаемся аллокацией виртуальных регистров. Реализован алгоритм «Linear Scan Register Allocation». Проводится статический анализ, при котором выделяются времена жизни все виртуальных регистров. Сортируем интервалы и дальше снова приходимся по всей программе, поддерживаем используемые регистры в данный момент времени и аллоцируем виртуальные регистры.
   - Если надо использовать какой-то виртуальный регистр, мы уже знаем где он должен храниться в физическом регистре, используем его.
   - Если в какой-то момент времени нужно больше виртуальных регистров, чем у нас есть вытесняем самый долгоживущий регистр на стек.
   - Если надо использовать какой-то виртуальный регистр, мы уже знаем где он должен храниться в на стеке, восстанавливаем его в специально зарезервированные регистры и используем.
6. Транслируем в машинный код

Проделана некоторая оптимизация. Если восстановленный регистр не используется для косвенной адресации его можно прочитать напрямую из памяти.

Также локальные переменные тоже часто не используются для косвенной адресации. Поэтому их тоже можно использовать на уровне операнда инструкции в виде косвенной адресации со смещением.

Вообще вся моя оптимизация свелась к использования mem-to-mem операций, там где это возможно. Это работает быстрее, так как работа с такими операциями оптимизирована на уровня тактов процессора.

4. Система команд

4.1. Инструкции с n операторов

- [31:25] - код оператора
- [24] - выбор размера слова (0 - байт, 1 - 8 байт)
- $[23-8i{:}16-8i]$ байт - i оператор

4.2. Инструкции переходов

- [63:55] - код оператора
- [54] - резерв
- [53:0] - адрес перехода

4.3. Описание операнда

- [7:5] - режим операнда
- [4:2] - основной регистр
- [1:0] - смещение задаётся как: 00 - следующие 32 бита константа, 01 - D5, 10 - D6, 11 - D7

4.4. Режим операнда

- 0x0 - #* - прямая загрузка
- 0x1 - D* - регистр данных
- 0x2 - A* - адресный регистр
- 0x3 - (A*) - косвенная
- 0x4 - (A*)+ - косвенная с постинкрементом
- 0x5 - -(A*) - косвенная с предекрементом
- 0x6 - (A:O) - косвенная со смещением
- 0x7 - (#*) - косвенная с прямой занрузкой

4.5. Коды операторов:

Обозначение	**Значение**
`#*`	Непосредственное значение
`D*`	Регистры данных (D0–D7)
`A*`	Адресные регистры (A0–A7)
`MEMORY`	Косвенная адресация
`.size`	`.b` (байт) / `.l` (слово)

Код	Мнемоника	Описание	Такты
0x0	HLT	Остановка выполнения процессора	2
0x01	MOV.size from, to	from := `#*` | `D*` | `A*` | `MEMORY` to := `D*` | `A*` | `MEMORY`	3 - 5
0x02	CMP.size that, with	that, with := `#*` | `D*` | `A*` | `MEMORY` Обновляет NZCV (that − with)	3 - 5
0x10	ADD.size from, to	from := `#*` | `D*` | `A*` | `MEMORY` to := `D*` | `A*` | `MEMORY` Обновляет NZCV	3 - 5
0x11	ADC.size from, to	from := `#*` | `D*` | `A*` | `MEMORY` to := `D*` | `A*` | `MEMORY` Обновляет NZCV и учитывает Carry	3 - 5
0x12	SUB.size from, to	from := `#*` | `D*` | `A*` | `MEMORY` to := `D*` | `A*` | `MEMORY` Обновляет NZCV	3 - 5
0x13	MUL.size from, to	from := `#*` | `D*` | `A*` | `MEMORY` to := `D*` | `A*` | `MEMORY` Обновляет NZCV	3 - 5
0x14	DIV.size from, to	from := `#*` | `D*` | `A*` | `MEMORY` to := `D*` | `A*` | `MEMORY`	3 - 5
0x15	REM.size from, to	from := `#*` | `D*` | `A*` | `MEMORY` to := `D*` | `A*` | `MEMORY` Обновляет NZCV	3 - 5

		Выставляет Carry, если деление на ноль	
0x16	AND.size from, to	from := #* | D* | A* | MEMORY to := D* | A* | MEMORY	3 - 5
0x17	OR.size from, to	from := #* | D* | A* | MEMORY to := D* | A* | MEMORY	3 - 5
0x18	XOR.size from, to	from := #* | D* | A* | MEMORY to := D* | A* | MEMORY	3 - 5
0x19	NOT.size from, to	from := #* | D* | A* | MEMORY to := D* | A* | MEMORY	3 - 5
0x1A	LSL.size count, dest	count := #* | D* | A* | MEMORY dest := #* | D* | A* | MEMORY Выставляет Carry	3 - 5
0x1B	LSR.size count, dest	count := #* | D* | A* | MEMORY dest := #* | D* | A* | MEMORY Выставляет Carry	3 - 5
0x1C	ASL.size count, dest	count := #* | D* | A* | MEMORY dest := #* | D* | A* | MEMORY	3 - 5
0x1D	ASR.size count, dest	count := #* | D* | A* | MEMORY dest := #* | D* | A* | MEMORY Работает с сохранением знака	3 - 5
0x20	JMP label	PC = label	2
0x21	CALL label	stack.push(PC + 1) PC = label	2
-	INTERRUPT	IF = 0 stack.push(PC) PC = INT_TABLE[vector] MEM[-(A7)] = NZCV	3
0x22	RET	PC = stack.pop()	2
0x23	INTRET	PC = stack.pop() NZCV = MEM[(A7)+] IF = 1	3
0x30	BEQ label	Переход по NZCV (Z=1)	2

0x31	BNE label	Переход по NZCV (Z=0)	2
0x32	BGT label	Переход по NZCV (Z=0 && N=O)	2
0x33	BGE label	Переход по NZCV (N=O)	2
0x34	BLT label	Переход по NZCV (N!=O)	2
0x35	BLE label	Переход по NZCV (Z=1 || N!=O)	2
0x36	BCS label	Переход по NZCV (C=1)	2
0x37	BCC label	Переход по NZCV (C=0)	2
0x38	BVS label	Переход по NZCV (O=1)	2
0x39	BVC label	Переход по NZCV (O=0)	2
0x40	VADD a, b	a, b := `A*`	5
0x42	VSUB a, b	a, b := `A*`	5
0x43	VMUL a, b	a, b := `A*`	5
0x44	VDIV a, b	a, b := `A*`	5
0x45	VREM a, b	a, b := `A*`	5
0x46	VAND a, b	a, b := `A*`	5
0x47	VOR a, b	a, b := `A*`	5
0x48	VXOR a, b	a, b := `A*`	5
0x49	VEND c	c := `A*` Запись результата векторной операции	3
0x50	IN port, dest	dest := `D*` | `A*` | MEMORY	2-3
0x51	OUT port, source	source := `#*` | `D*` | `A*` | MEMORY	2-3
0x52	EI	Включает прерывания	2
0x53	DI	Выключает прерывания	2
0x60	VCMPBEQ a, b	a, b := `A*` Выводит маску из «1»	5
0x61	VCMPBNE a, b	a, b := `A*` Выводит маску из «1»	5
0x62	VCMPBGT a, b	a, b := `A*` Выводит маску из «1»	5
0x63	VCMPBGE a, b	a, b := `A*` Выводит маску из «1»	5
0x64	VCMPBLT a, b	a, b := `A*` Выводит маску из «1»	5

0x65	VCMPBLE a, b	a, b := `A`* Выводит маску из «1»	5
0x66	VCMPBCS a, b	a, b := `A`* Выводит маску из «1»	5
0x67	VCMPBCC a, b	a, b := `A`* Выводит маску из «1»	5
0x68	VCMPBVS a, b	a, b := `A`* Выводит маску из «1»	5
0x69	VCMPBVC a, b	a, b := `A`* Выводит маску из «1»	5

* Там где не указано точное количество тактов, исполнения инструкции, значит что она имеет переменное число тактов. Чем больше косвенной адресации, тем больше тактов.

5. Транслятор

6. Управляющее устройство

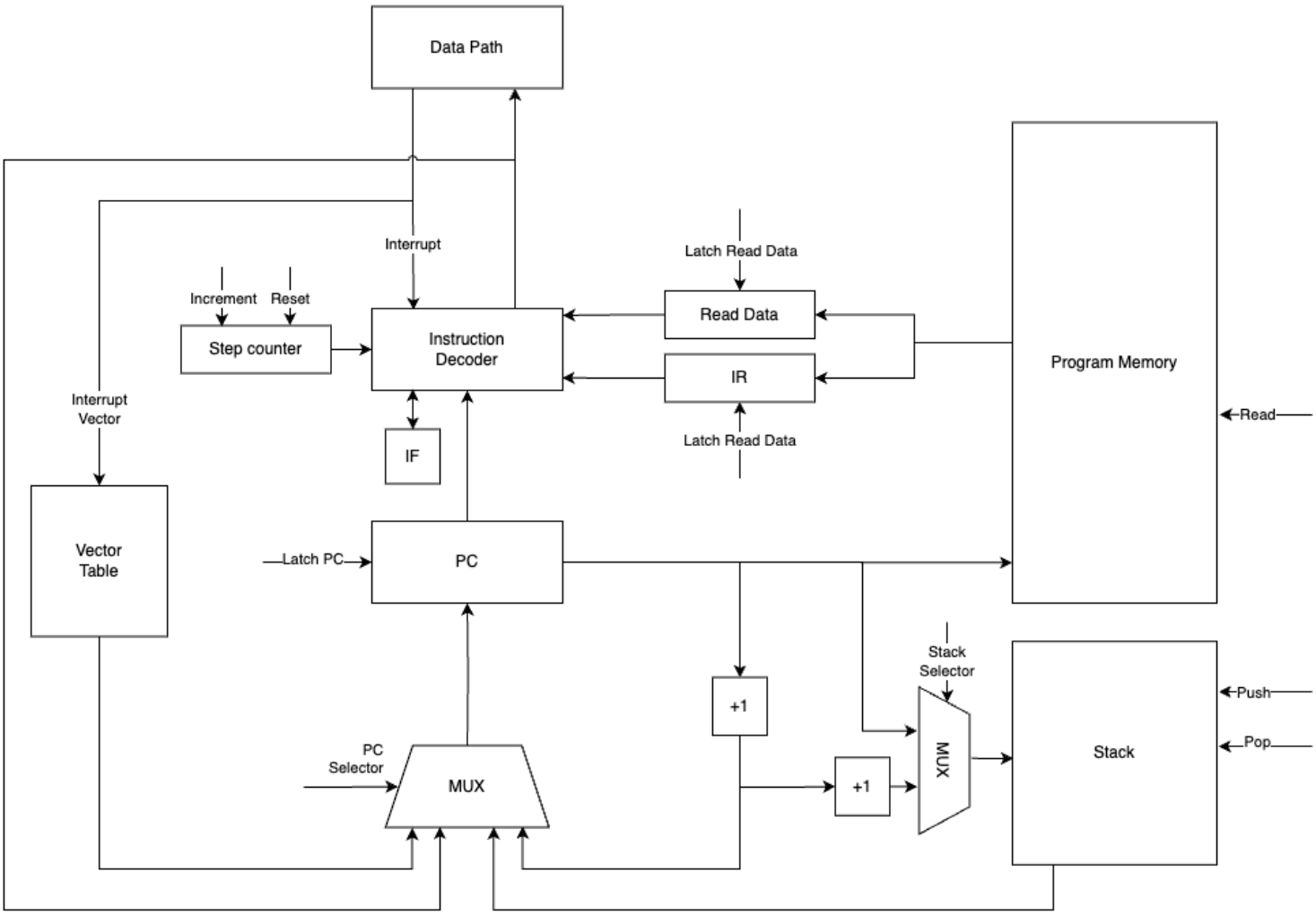
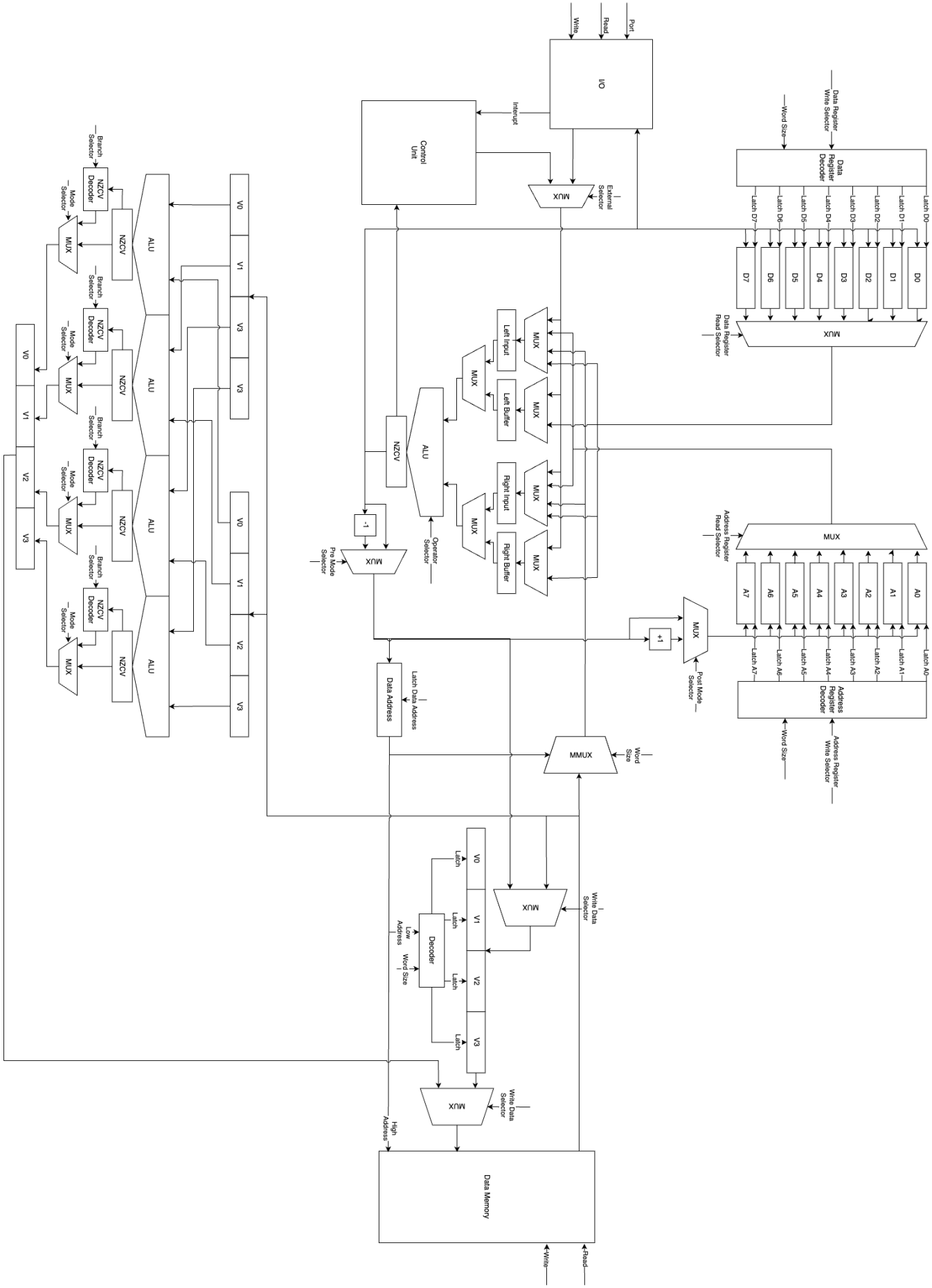


Рис. 1. Схема управляющего устройства

7. Тракт данных



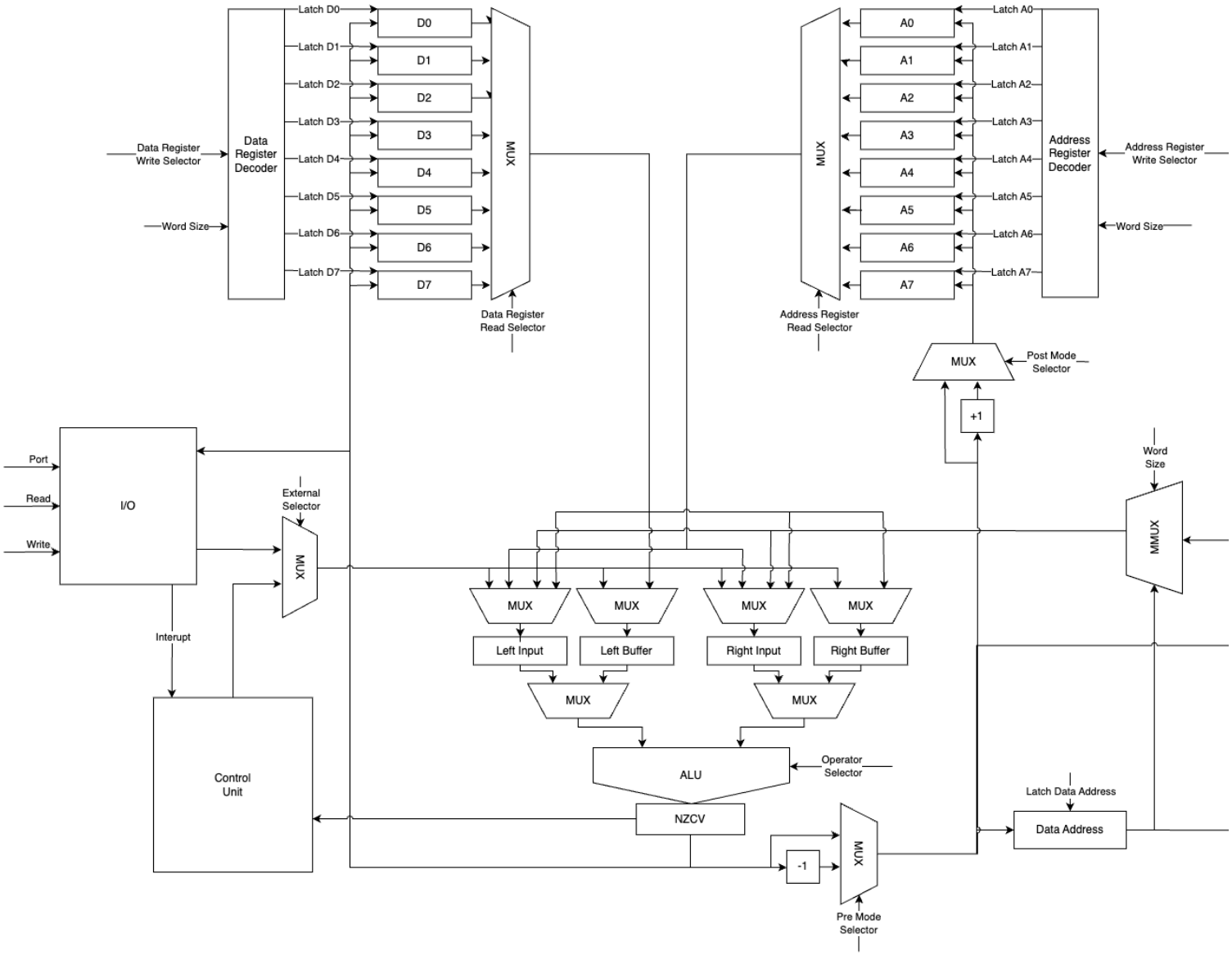


Рис. 3. Схема АЛУ

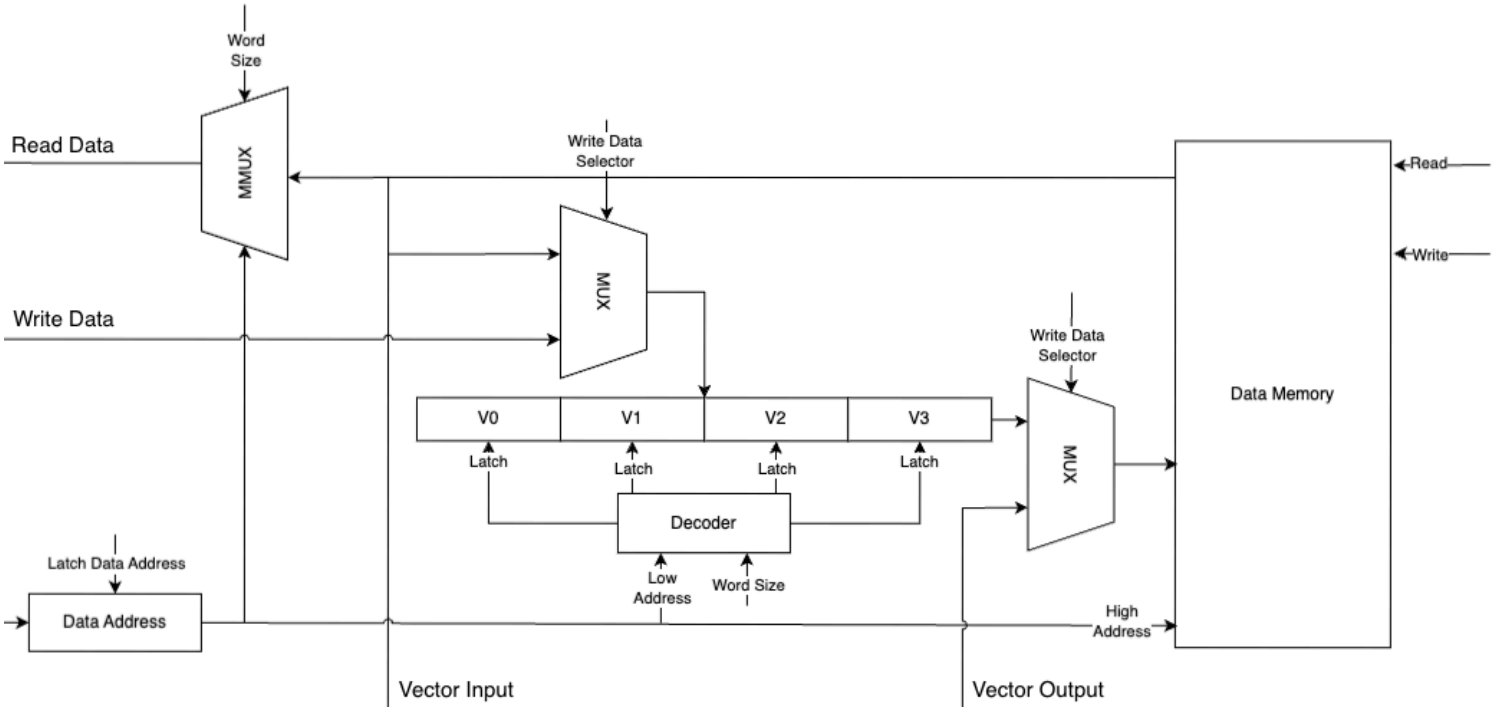


Рис. 4. Схема памяти

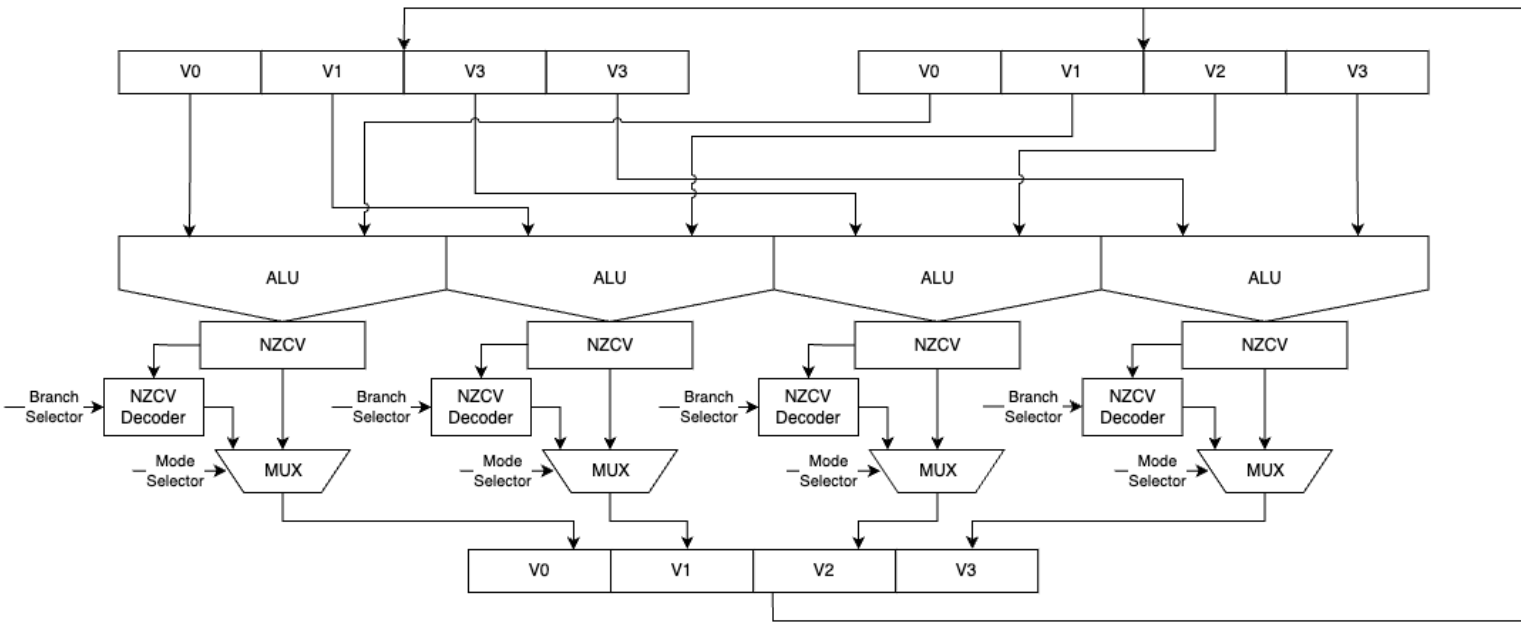


Рис. 5. Схема векторного АЛУ

8. Векторные операции

8.1. Обшая информация

Для поддержки SIMD операций был добавлен дополнительный вычислительный блок. Он содержит 3 256-битных регистра; 4 независимые линии, каждая из которых содержит свой АЛУ. После выхода АЛУ есть схемотехника, которая может выставить маску в зависимости от NZCV конкретного АЛУ и вида сравнения.

Дизайн работы с памятью обусловлен SIMD операциями. Такие операции будут очень упираться в пропускную способность памяти. Поэтому был выбран большой размер слова. Так же стоит отметить что хотя запись в память и работает за два такта, но на общее время выполнения инструкции это не повлияло, так как это задача может выполняться параллельно.

8.2. Сложение векторов

В файлах «vector_test» и «vector_test_simd» представлены программы суммирующие два массива и складывающие результат в другой массив.

Это задача нигде не требует синхронизаций, поэтому хорошо параллелится. Как видно из таблицы ниже производительность SIMD операций в таком случае близка к теоретической.

	Внешние операции	Скаляр	SIMD
Такты	19859	28185	22010
Норма	0	8326	2151
Скорость	-	100%	387%

8.3. Перемножение матриц

В файлах «matrix» и «matrix_simd» представлены программы перемножающие две матрицы 8x8.

Также я выбрал задачу перемножения матриц как ту, которая сможет показать реальную картину.

	Внешние операции	Скаляр	SIMD
Такты	12767	70729	51785
Норма	0	57962	39018
Скорость	-	100%	148%

При реализации алгоритма пришлось порождать новые массивы, чтобы воспользоваться векторизацией, поэтому прирост производительности не такой большой.